

When Learned IDS Model Selection Collapses to a Static Choice: A Measurement Study at the Edge

Nuonan Ouyang, Adrian Shatte, Zhigang Lu, Chao Chen, *Member, IEEE*, and Wei Xiang, *Senior Member, IEEE*

Abstract—Learned model selection can appear to save energy when it merely fixes a cheaper detector. On one Raspberry Pi 4B, primary Tabular-Q uses 13.1 J less than Static-MedRF over 500 s (1.08%; 26.1 mW), but missed-positive presentations rise from 57.8 to 565.9 per 50,000-row replay and the policy selects LightLR throughout. A matched Static-LightLR control resolves no directional Tabular-Q energy difference. We derive a pairwise non-degeneracy test showing that the executed reward scaling requires a 0.7967 balanced-accuracy advantage for MedRF to outrank LightLR before held-out evaluation. Under a prospectively frozen 50/100/4000 rows/s $\times$ 0.10/0.50/0.90 factorial, DeadlineGuard switches between MedRF and LightLR, uses 43.7 J less than Static-MedRF (2.98%; 95% paired CI 41.4–46.0 J), and eliminates 512/5,000 deadline misses at a 1.373-pp F1 cost. Frozen DQN remains LightLR, whereas Tabular-Q falls to TinyDT (67.76% FPR). The results separate detector choice, conditional adaptation, reward specification, and off-support controller behavior in edge IDS orchestration.

Index Terms—Intrusion detection, IoT, edge resource management, model selection, reinforcement learning, energy measurement, reproducibility.

I. INTRODUCTION

Edge intrusion detection must balance attack recognition with latency and energy. Runtime selection appears attractive, but a learned controller may simply choose one detector throughout deployment. Comparing it only with a more expensive static detector then measures an operating-point change rather than adaptive value. In our primary campaign, Tabular-Q saves 13.1 J over roughly 500 s relative to Static-MedRF—1.08%, or 26.1 mW on average—while missed-positive presentations rise from 57.8 to 565.9 per 50,000-row replay. This scale mismatch motivates a stricter question: *what schedule was actually realized, under what service constraints, and at what security cost?*

(Corresponding author: Nuonan Ouyang.)

N. Ouyang and A. Shatte are with the College of Science and Engineering, Division of Research, James Cook University, Bebegu Yumba Campus, Building 17, Douglas, Townsville, QLD, Australia (e-mail: nuonan.ouyang@my.jcu.edu.au; adrian.shatte@jcu.edu.au; ORCID: 0009-0004-7124-8940; 0000-0002-6225-9697).

Z. Lu is with the School of Computer, Data and Mathematical Sciences, Western Sydney University, Penrith (Kingswood) Campus, Kingswood, NSW, Australia (e-mail: z.lu@westernsydney.edu.au; ORCID: 0000-0001-5102-6217).

C. Chen is with the College of Business and Law, RMIT University, Melbourne, VIC 3000, Australia (e-mail: chao.chen@rmit.edu.au; ORCID: 0000-0003-1355-3870).

W. Xiang is with the Australian Centre for AI in Medical Innovation and the Cisco-La Trobe Centre for AI and Internet of Things, La Trobe University, Melbourne, VIC, Australia (e-mail: w.xiang@latrobe.edu.au; ORCID: 0000-0002-0608-065X).

We answer that question as a TON-IoT-centered systems study with four frozen detectors and paired Raspberry Pi measurements; CICIoT2023 and N-BaIoT remain supplementary detector/data audits rather than dynamic-power evidence. The evidence separates the primary physical campaign and Static-LightLR attribution control, an analytic reward non-degeneracy screen, and prospectively frozen confidence/deadline-aware controls under fixed and variable load.

Four questions organize the paper: *RQ1*, what security/resource operating points do the frozen TON-IoT detectors provide? *RQ2*, do learned selectors improve upon the static schedule they realize, and is their scalar objective non-degenerate on the declared support? *RQ3*, do confidence- and deadline-aware conditional controls provide useful switching? *RQ4*, how do state coverage, off-support actions, deadline violations, and threshold calibration constrain deployment claims?

Our contributions are: (i) paired whole-device evidence that places a small resource effect beside its much larger missed-detection cost; (ii) a pairwise non-degeneracy criterion that converts reward weights and normalization into a pre-deployment preference-reversal test; (iii) static, learned, confidence-cascade, and variable-load controls that separate detector choice from conditional orchestration; and (iv) an off-support failure analysis showing how sparse discrete-state coverage can expose unsafe default actions. Together, the study identifies when learned IDS orchestration does—and does not—realize adaptive service value.

II. RELATED WORK AND EVALUATION PERSPECTIVE

A. Dynamic IDS Selection and Conditional Inference

Recent IDS work spans continual/incremental detection, heterogeneous representations, and runtime model selection [1], [2], [3], [4]. Our detector artifacts are frozen; the runtime problem is which detector to execute. Auto-IDaS uses DQN for detector selection under traffic variation, whereas FEDMS uses confidence-driven dynamic selection [5], [6]. These establish plausible adaptive mechanisms, but not that a learned selector provides value beyond the static action it realizes.

Conditional computation provides a direct non-RL alternative. BranchyNet, adaptive network selection, MSDNet, and shallow-deep networks use confidence or budget signals to invoke more computation only for harder inputs [7], [8], [9], [10]. At the serving layer, Clipper, INFaaS, and Clockwork expose accuracy/latency/cost trade-offs and request-level

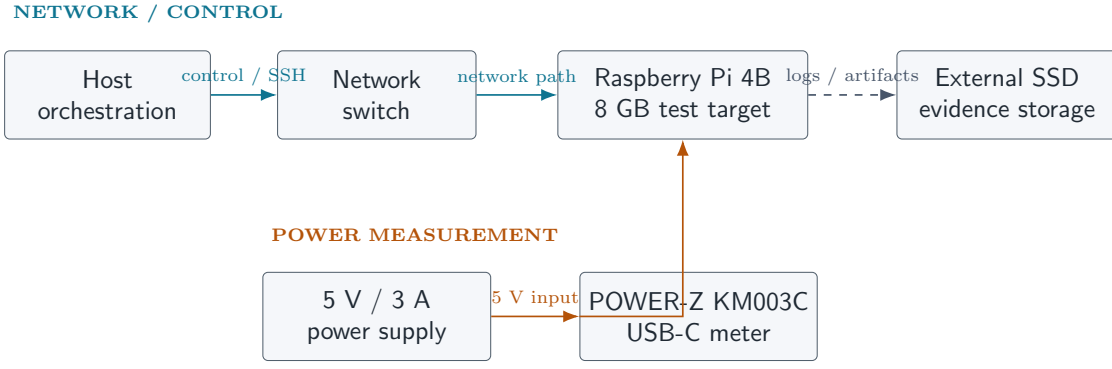


Fig. 1. Schematic apparatus overview. Network orchestration reaches the Raspberry Pi 4B through the switch, while the 5-V supply feeds the Pi through the KM003C meter; the external SSD stores evidence artifacts. The drawing identifies roles and paths only and contains no synthetic measurement readout.

SLOs [11], [12], [13]. These motivate our confidence cascade and deadline-aware comparator.

B. Learned Resource Orchestration and Evaluation

Reinforcement learning is widely used for runtime resource orchestration. TNSM examples include measurement-informed vRAN reconfiguration and vehicle-edge scheduling [14], [15]; related mobile/edge systems couple DVFS, dependent-task scheduling, offloading, and QoS control to latency or energy objectives [16], [17], [18], [19], [20], [21], [22], [23], [24]. Tabular Q-learning estimates values in a discrete state space [25], while DQN uses a neural action-value approximator [26]. Controller complexity, however, is not evidence of useful adaptation.

Security-ML evaluations can be optimistic even when implementations are internally correct [27]. We therefore expose source/encoded-data identity, confusion counts, realized actions, deadlines, and raw meter frames, and keep training variability, replay resampling, and physical repeatability as separate uncertainty sources [28]. The block—not rows, windows, or meter samples—is the physical repeat unit.

III. DATASET-NATIVE DATA PREPARATION

The main systems study uses TON-IoT. Its frozen pipeline retains 32 native features and admits no feature into a cross-dataset core. The TON-IoT split groups unordered IP pairs, fits imputation/scaling/categorical encoding on training data only, and uses training seed 11. Source-row identifiers and raw-field fingerprints are cross-partition disjoint; the independent encoded-float32 audit finds zero TON-IoT train-test vector matches. Exact counts, lineage, and split membership are supplementary.

The same ledger also records dataset-native CICIOT2023 and N-BaIoT pipelines (39 and 115 native features), but these are supplementary detector/data audits only. Their coverage and encoded-overlap limitations are retained rather than used to imply cross-dataset dynamic-power generality [29].

IV. FROZEN DETECTORS AND CAUSAL SCHEDULING

A. Detector Pool

Each dataset has four separately fitted configurations: depth-6 TinyDT; L2 LightLR; 25-tree, depth-10 MedRF; and a

64/32/16 HeavyMLP. All use threshold 0.5 and training seed 11. Names do not assert a resource ranking. During TON-IoT physical experiments all models remain resident; an action selects one model for the next batch without loading, fitting, or updating it.

B. Decision Information and State Encoding

Before window t , the controller constructs

$$s_t = (\tilde{T}_t, h_{t-1}, q_t, \lambda_{t-1}, a_{t-1}). \quad (1)$$

The kernel temperature is smoothed as $\tilde{T}_t = 0.9\tilde{T}_{t-1} + 0.1T_t$, initialized at T_0 . The threat observation h_{t-1} is the arithmetic mean attack probability from the single detector selected in the preceding window; it is not the preceding ground-truth attack fraction. Queue utilization is bounded to $[0, 1]$, and the preceding arrival count is divided by 100 and bounded to $[0, 2]$. Initial threat, arrival ratio, and preceding action are 0.5, 1, and TinyDT.

Each of the first four state variables has three bins. Their two boundaries are (45, 70) degrees Celsius, (1/3, 0.7) for threat, (1/3, 2/3) for queue utilization, and (0.9, 1.1) for arrival ratio. Including four previous actions gives $3^4 \times 4 = 324$ states. Boundary equality enters the higher bin. The flat index is row-major in this listed order, with previous action changing fastest.

The executed order is state, committed/logged action, selected-model inference, outcome log, then lagged-state update. Ground truth, cell identity, and unselected current-window outputs are unavailable online. Original software traces use 35°C, zero queue, and 100 arrivals/window; the variable-load experiment varies causal arrival count and prevalence without revealing factorial-cell identity.

C. Reward and Cost Provenance

The unchanged per-window training reward is

$$r_t = 0.5B_t - 0.2\bar{P}_t - 0.2\bar{L}_t - 0.1\bar{H}_t, \quad (2)$$

where B_t is balanced accuracy over classes present in the window. All evaluated phase windows contain both classes. With $[x]_0^1 = \min(1, \max(0, x))$, the normalization is

$$\bar{P}_t = \left[\frac{P_t - 2.347219336}{0.049806200} \right]_0^1, \quad (3)$$

$$\bar{L}_t = \left[\frac{L_t - 1.2979595}{29.6547998} \right]_0^1, \quad (4)$$

$$\bar{H}_t = \left[\frac{\max(0, T_t^{\text{post}} - \tilde{T}_t)}{1^\circ\text{C}} \right]_0^1. \quad (5)$$

P_t and L_t are frozen TON-IoT paced-profile lookups, not contemporaneous training measurements. Their means are 2.353132/1.297960, 2.347219/1.544420, 2.397026/30.952759, and 2.366688/4.417836 W/ms for TinyDT, LightLR, MedRF, and HeavyMLP. No fitted thermal response or switching-energy penalty is used.

D. Pre-deployment Pairwise Non-degeneracy Check

The scalar objective can be screened before policy evaluation. For two actions a, b , let $D_a(s)$ denote the detection term and let $C_a = \sum_j w_j \bar{C}_{j,a}$ collect normalized resource penalties that are fixed on the declared support (as in the executed training profiles at neutral thermal cost). Then

$$U_a(s) - U_b(s) = w_D \Delta D_{ab}(s) - \Delta C_{ab}, \quad \tau_{ab} = \frac{\Delta C_{ab}}{w_D}. \quad (6)$$

A strict pairwise preference reversal is possible on support S iff

$$\min_{s \in S} \Delta D_{ab}(s) < \tau_{ab} < \max_{s \in S} \Delta D_{ab}(s). \quad (7)$$

If τ_{ab} lies outside this attainable interval, no optimizer can make the pair exchange rank under the frozen scalar utility on that support. For MedRF versus LightLR, the declared min-max scaling gives $\tau = 0.7967$: an almost 80-percentage-point balanced-accuracy advantage would be required to offset the frozen nonthermal resource penalty. This threshold is determined only by the declared weights and resource profiles and can therefore be inspected before held-out evaluation; whether reversal is attainable is then checked against calibration/validation support rather than test outcomes. Candidate-pool min-max denominators make this exchange rate change whenever pool membership changes. For deployment, external denominators such as idle-relative power budgets, latency SLOs, or thermal headroom preserve a physically interpretable exchange rate. The proof and numerical decomposition are supplementary.

E. Policies and Offline Selection

CFSM is a fixed heuristic initialized to TinyDT. At a smoothed temperature of at least 70 degrees it steps down one model rank, bypassing cooldown. Otherwise it holds for two complete windows after a switch; when no cooldown remains, threat below 0.7 steps down one rank and threat at least 0.7 steps up one rank. Boundary actions hold. Queue and load are logged but not used by CFSM. This fixed ordering is evaluated as implemented, not asserted to be optimal.

Tabular-Q uses zero initialization, learning rate 0.1, discount 0.9, and the update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + 0.1 \delta_t, \quad \delta_t = r_t + 0.9 \max_a Q(s_{t+1}, a) - Q(s_t, a_t), \quad (8)$$

with zero terminal bootstrap [25]. DQN uses one-hot state, two 64-unit ReLU layers, Adam 0.001, replay 10,000, batch 32, and a target copy every 250 updates [26]. Both train for 1,000 episodes with epsilon 1 to 0.01. Checkpoints are selected every 25 episodes on five validation traces; test execution freezes the policy and performs no update. Full training, tie rules, seeds, and later multi-seed/sensitivity results appear in the supplement.

V. COMPLETED EVALUATION AND MEASUREMENT METHODS

A. Classification and Static Evidence

The held-out classification evaluation uses the frozen threshold and reports attack-class F1, recall, balanced accuracy, false-positive rate, and the full confusion counts. In particular,

$$\text{FPR} = \frac{FP}{TN + FP}, \quad (9)$$

$$\text{BA} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right), \quad (10)$$

$$\text{F1} = \frac{2TP}{2TP + FP + FN}. \quad (11)$$

Counts are pooled over the declared evaluation unit before metrics are calculated. The supplement retains 50 static traces and 200 model-trace evaluations. These reuse fixed held-out pools and do not constitute fresh attacks, classifier retraining, or physical repeats.

B. Pi Prediction Consistency and Inference Timing

Pi parity checks cover 12 dataset-model combinations and 10,000 encoded rows each with zero threshold-label disagreements. Tight-loop timing reports 100-row inference after warm-up and excludes encoding, transport, queueing, and controller cost.

C. Static Whole-Device Power

The physical reference uses one Pi 4B 8 GB and inline POWER-Z KM003C. Fifteen valid 1,800-second segments comprise three repeats of instrumented idle and four TON-IoT detectors, all resident, at 100 preencoded rows/s. This is resource characterization, not another detection test.

The meter records nominally 1 Hz boundary samples, including raw ADC replies, signed current, voltage, and monotonic request/receive times. Under the recorded connector orientation, consumed power is calculated as $P_i = V_i(-I_i)$. Using the observed sampling times τ_i , input energy is approximated by

$$E = \sum_{i=1}^{n-1} \frac{P_i + P_{i+1}}{2} (\tau_{i+1} - \tau_i), \quad \bar{P} = \frac{E}{\tau_n - \tau_1}. \quad (12)$$

No missing sample is filled. This is 1-Hz load-side USB energy, not calibrated wall-side measurement or a subsecond waveform.

D. Statistical Units and Interval Reporting

For an outcome z observed over n declared repetitions, the two-sided interval is

$$\bar{z} \pm t_{0.975, n-1} \frac{s_z}{\sqrt{n}}, \quad (13)$$

using the sample SD [30]. Physical scheduler inference uses ten paired blocks, never rows, windows, or meter frames. Raw energy is primary. Drift sensitivity subtracts each run's preceding idle power; it is reported separately rather than substituted for raw measurement. Exact 2^{10} sign-flip tests are secondary diagnostics.

E. Physical Campaigns

The primary campaign contains ten matched 500-second blocks of Static-MedRF, CFSM, Tabular-Q, and DQN. matched-control subsequently adds ten blocks with the missing Static-LightLR control; scheduler-overhead isolates state-to-action timing; multi-seed-hyperparameter sensitivity test seeds, reward/state variants, and Tabular-Q α/γ . They do not replace the frozen primary policy.

A fixed-rate confidence-cascade control uses ten new matched blocks at 100 rows/s with Static-LightLR, a fixed confidence cascade, frozen Tabular-Q, and Static-MedRF. The cascade first runs LightLR and escalates a row to MedRF iff LightLR attack probability is in the fixed band $[0.3, 0.7]$.

A variable-load campaign uses ten blocks and six methods over 500 seconds. Each run visits the nine predeclared 50-second cells $\{50, 100, 4000\}$ rows/s $\times \{0.10, 0.50, 0.90\}$ attack prevalence exactly once, followed by 50 seconds at 100/0.10. DeadlineGuard uses only current causal arrival count: MedRF is chosen when $\lceil N/100 \rceil L_{95}^{\text{RF}} \leq 1$ s, otherwise LightLR. The threshold and workload were frozen from previously measured latency. Completion timestamps define a one-second service deadline; queue, states, actions, fallbacks, and misses are retained.

controller-only power arm predeclared controller-only physical power over four "original observed-support states." The frozen artifact contains eight such states and no rule uniquely identifies four; under the stop rule controller-only power arm is NOT_EXECUTED. TinyDT threshold diagnostic chooses a TinyDT threshold on validation by maximum balanced accuracy (highest-threshold tie break), hashes it, and applies it once to test. This diagnostic never replaces threshold 0.5.

VI. COMPLETED STATIC RESULTS

A. TON-IoT Detection Operating Points

Table I shows the four TON-IoT operating points used by the physical study. TinyDT has recall 0.9976 but F1 0.4792 and FPR 67.78%; LightLR, MedRF, and HeavyMLP have F1 0.9620–0.9746 and FPR 1.35–1.52%. Full confusion counts, plus CICIoT2023/N-BaIoT detector and overlap audits, are retained in the supplement rather than used to imply cross-dataset dynamic-power generality.

For attack prevalence p , recall r , and FPR f ,

$$F1 = \frac{2pr}{p(1+r) + (1-p)f}, \quad (14)$$

TABLE I
TON-IoT HELD-OUT DETECTION AT THRESHOLD 0.5. BA DENOTES
BALANCED ACCURACY; FPR IS A PERCENTAGE.

Model	F1	Recall	BA	FPR (%)
TinyDT	0.4792	0.9976	0.6599	67.78
LightLR	0.9620	0.9668	0.9766	1.35
MedRF	0.9746	0.9965	0.9906	1.52
HeavyMLP	0.9736	0.9903	0.9883	1.38

so high attack prevalence can coexist with high F1 and poor normal-class behavior. Static replay and encoded-overlap diagnostics are supplementary and do not change the fixed split, model, or primary threshold.

B. Static Physical Resource Measurements

Across fifteen static segments, idle averages 2.335 W and active means span 2.347 W (LightLR) to 2.397 W (MedRF). MedRF's paired increment is 61.6 mW [32.1, 91.0]; TinyDT and LightLR intervals cross zero. No accepted workload window records undervoltage or throttling. Full segment statistics are supplementary; labels such as "Tiny" or "Heavy" are not used as empirical cost ranks.

VII. PHYSICAL SCHEDULER RESULTS

A. Primary Attribution and Robustness

All forty primary runs contain 501 power samples and 500 decisions. Relative to Static-MedRF, Tabular-Q saves 13.068 J (95% paired CI 9.736–16.400) and DQN saves 6.227 J (1.600–10.853), but both lose 1.343 F1 pp and 2.954 recall pp. Both select LightLR in all 5,000 windows; thus the result does not demonstrate switching.

The matched Static-LightLR control directly compares the learned policies with Static-LightLR. Tabular-Q has identical predictions and +0.455 J with CI $[-1.861, 2.772]$ J; DQN has identical predictions and +4.525 J [2.693, 6.357]. The isolated state-to-action benchmark time is 10.8 μ s for Tabular-Q and 1.567 ms for DQN, but timing is not converted into joules. Twenty policy seeds, 85 reward/state runs, and 35 Tabular-Q α/γ runs remain single-action on held-out support; only the detection-only objective selects MedRF rather than LightLR. Full primary and strengthening tables are supplementary.

B. Fixed-Rate Confidence Cascade

Table II reports ten new matched blocks. The fixed cascade escalates 1.34% of rows and raises F1 by 1.347 pp versus Static-LightLR while using +7.952 J [5.897, 10.007]. Its raw mean-power difference is +15.917 mW [11.798, 20.035], whereas preceding-idle adjustment gives +13.635 mW $[-1.515, 28.784]$; environmental drift therefore weakens the power attribution but not the raw-energy result. Cascade F1 (0.98391) is close to Static-MedRF (0.98386), with 0.279 lower recall pp and 0.156 lower FPR pp.

The cascade latency is implementation- and call-granularity-specific. Frozen executed source first runs LightLR on the full 100-row batch and, when needed, makes *one* MedRF call

on the deferred sub-batch; it does not call MedRF once per deferred row. Although only 1.3388% of rows are deferred, they are spread across 3,338/5,000 windows (66.76%), with only 2.01 deferred rows per active window. Windows without escalation average 1.636 ms, whereas escalation-active windows average 31.884 ms, close to the cost of one MedRF call; the resulting overall cascade mean is 21.830 ms. Thus This campaign measures this 100-row, small-sub-batch implementation regime and is not a general energy or latency bound for optimized cascade systems.

C. Variable Load, Feasibility, and State Coverage

The variable-load campaign visits all nine frozen load/prevalence cells in every run. DeadlineGuard executes MedRF in 3,500 and LightLR in 1,500 of its 5,000 windows, with 5.3 recorded switches/run, and misses no service deadline. Static-MedRF misses 512/5,000 deadlines (10.24%, block CI 9.32–11.16%) although its final backlog returns to zero. Cascade misses 21/5,000; all other methods miss none.

The strongest paired comparison is DeadlineGuard against Static-MedRF. DeadlineGuard uses 43.714 J less per run (95% paired CI 41.418–46.010 J), a 2.98% reduction relative to the 1468.1-J Static-MedRF mean, while reducing the deadline-miss rate by 10.24 pp to zero. The trade-off is 1.373 pp lower F1 and 2.853 pp lower recall. All ten energy differences have the same direction (exact sign-flip $p = 0.00195$). This establishes a measured feasibility/energy benefit for the simple causal rule relative to the high-security static baseline, not universal superiority over all static choices.

The frozen learned policies do not provide useful load adaptation. DQN remains LightLR in all 5,000 windows and reproduces its detection. Tabular-Q reaches new load-related states and selects TinyDT in every window through the frozen tabular default, yielding F1 0.7434 with FPR 67.76%. Here “fallback” is registry terminology, not an exception branch. On unseen zero-initialized rows, `np.argmax` returns the first maximum and the configured action order places TinyDT first. Thus the variable-load failure is the hardware realization of the off-support tie behavior already visible in the 314/324-state TinyDT greedy map.

The fixed heuristic provides a switching contrast rather than a no-op baseline. CFMS averages 9.8 switches per run, but spends 74.20% of windows on TinyDT and only 22.26% on HeavyMLP (LightLR 2.28%, MedRF 1.26%). Its switching changes detector identity without reaching a reliable security operating point; the low F1 is not evidence that CFMS simply selected one static model.

Variable-load raw energy is heterogeneous across methods, so Table III now reports run SDs and selected paired contrasts are separated in Table IV. For DQN versus Static-LightLR, all ten raw block differences are positive (median +5.72 J; exact sign-flip $p = 0.00195$), but one Block 4 excursion is +392.0 J and inflates the SD of differences to 122.2 J, giving a Student- t interval of $[-43.2, 131.7]$ J. That run coincides with a 3.919-W preceding idle and a 49.2°C maximum temperature, unlike the other DQN blocks; idle-adjusted power does not resolve a DQN–LightLR difference. We therefore report a consistent raw

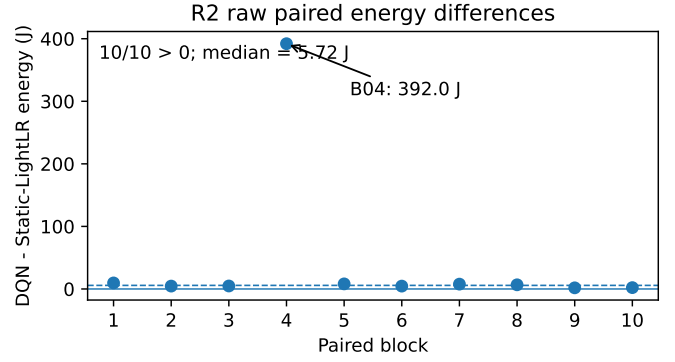


Fig. 2. Raw DQN-minus-Static-LightLR energy difference for each paired block. All ten are positive, but Block 4 is a large power/temperature excursion; the dashed line is the median (5.72 J).

R2 operational signature across the frozen factorial

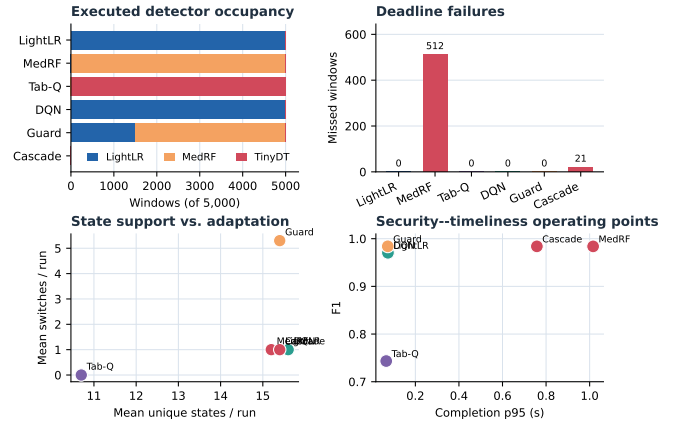


Fig. 3. Variable-load operational dashboard: realized detector occupancy, deadline misses, state support/switching, and security–timeliness operating points in the same frozen 60-run factorial.

direction but an unstable magnitude and make no controller-power attribution because the controller-only power arm was not executed.

For scale only, the isolated DQN state-to-action timing is 1.5669 ms. Five thousand decisions therefore amount to 7.8345 s of controller CPU time. If an external device-power increment were hypothetically 0.2, 0.5, or 1.0 W, the corresponding upper-bound contributions would be 1.57, 3.92, and 7.83 J, respectively. These are sensitivity calculations, not measurements: no controller-power increment was measured and none is used to attribute the physical DQN premium.

DeadlineGuard uses +7.550 J [5.033,10.068] and +14.160 adjusted mW [0.374,27.947] relative to Static-LightLR. Static-MedRF uses +51.264 J [49.537,52.991] and +98.609 adjusted mW [79.296,117.922] relative to LightLR. Cascade improves F1 by 1.384 pp but its raw +31.262 J interval crosses zero; the idle-adjusted +98.843 mW interval [77.885,119.802] does not. Raw and preceding-idle-adjusted endpoints are retained separately because they answer different questions.

TABLE II

FIXED-RATE CASCADE PHYSICAL RESULTS; MEANS OVER TEN PAIRED BLOCKS. ADJUSTED POWER SUBTRACTS THE IMMEDIATELY PRECEDING IDLE. ESCALATION IS THE FRACTION SENT FROM LIGHTLR TO MEDRF.

Method	Energy (J)	Power (W)	Adjusted (mW)	F1	Recall (%)	FPR (%)	Escalation (%)
Static-LightLR	1360.4	2.721	37.0	0.9704	96.71	1.36	–
Cascade	1368.3	2.737	50.6	0.9839	99.39	1.38	1.34
Tabular-Q	1362.0	2.724	39.7	0.9704	96.71	1.36	–
Static-MedRF	1368.6	2.737	56.5	0.9839	99.66	1.54	–

TABLE III

VARIABLE-LOAD RESULTS OVER TEN PAIRED BLOCKS. ENERGY IS MEAN $\pm$ SD ACROSS RUNS; DEADLINE MISS IS THE FRACTION OF ONE-SECOND WINDOWS MISSING THE SERVICE DEADLINE.

Method	Energy (J)	Power (W)	F1	Recall (%)	FPR (%)	Miss (%)	Completion p95 (s)
Static-LightLR	1416.8 $\pm$ 3.7	2.834	0.9763	96.68	1.35	0.00	0.070
Cascade	1448.1 $\pm$ 46.4	2.896	0.9902	99.41	1.36	0.42	0.665
DeadlineGuard	1424.4 $\pm$ 5.2	2.849	0.9769	96.80	1.36	0.00	0.069
Tabular-Q	1400.9 $\pm$ 48.0	2.802	0.7434	99.76	67.76	0.00	0.063
DQN	1461.1 $\pm$ 123.9	2.922	0.9763	96.68	1.35	0.00	0.072
Static-MedRF	1468.1 $\pm$ 4.8	2.936	0.9906	99.65	1.52	10.24	1.013

TABLE IV

SELECTED VARIABLE-LOAD PAIRED CONTRASTS, $n = 10$ BLOCKS. ΔE IS FIRST METHOD MINUS SECOND; DETECTION AND DEADLINE CHANGES ARE PERCENTAGE POINTS.

Contrast	ΔE (J), 95% CI	$\Delta F1$	$\Delta Miss$	sign-flip p
DeadlineGuard – Static-MedRF	–43.7 [–46.0, –41.4]	–1.373	–10.24	0.00195
DQN – Static-LightLR	+44.2 [–43.2, +131.7]	+0.000	+0.00	0.00195

TABLE V

TON-IOT TINYDT TEST: FIXED PRIMARY VERSUS POST-HOC VALIDATION-SELECTED THRESHOLD.

Result	τ	TN	FP	FN	TP	F1	Rec.	FPR	BA
Primary fixed	0.500	5519	11610	13	5348	0.4792	99.76	67.78	0.6599
Validation-selected diagnostic	0.646751	16817	312	21	5340	0.9698	99.61	1.82	0.9889

D. Protocol Stop and TinyDT Threshold Diagnostic

The planned controller-only power arm is not missing data: it is a protocol-preserving stop. Its design required four original observed-support states, but the frozen registry exposes eight and supplies no pre-outcome selection rule. Choosing four afterward would modify the protocol, so no controller-only power segment was run and no DQN joule attribution is made.

The threshold diagnostic selects threshold 0.646751 on validation and applies it once to test. At threshold 0.5, TinyDT test F1/FPR/recall are 0.4792/67.78%/99.76%; the diagnostic yields 0.9698/1.82%/99.61%, with balanced accuracy 0.9889 and AUROC 0.9906. The large recovery with near-constant recall is consistent with a shifted decision threshold rather than absent ranking information. It does *not* make TinyDT an acceptable substitute in the frozen physical study: every classifier and scheduler artifact in those campaigns was fixed at threshold 0.5, and the post-hoc threshold cannot retroactively rehabilitate the unsafe TinyDT actions. The diagnostic therefore explains calibration sensitivity without replacing any primary or physical result.

The same validation-only procedure was also applied offline

to the other frozen TON-IoT detector scores. Table VI shows the resulting test diagnostics; no detector was retrained and no physical campaign was changed. TinyDT is the clear F1/FPR outlier: its validation-selected threshold changes test F1 by +49.05 points, whereas the other detectors change by at most 0.29 points. This separates calibration sensitivity from the sparse-state and deterministic-tie mechanism.

VIII. DISCUSSION

A. Attribution, Adaptation, and Operational Scale

The primary learned energy reduction is mainly detector choice: both learned policies execute LightLR, and the matched control resolves no Tabular-Q saving or premium against direct LightLR. Its 13.1 J contrast with Static-MedRF is only 26.1 mW on average while missed-positive presentations increase almost tenfold. At the primary 100 rows/s cadence, detector inference occupies only a small fraction of each one-second interval, so background platform power dominates the whole-device endpoint. Energy alone would therefore overstate the practical value of the learned policy.

The variable-load campaign supplies the positive service-management result that the primary campaign lacked. Relative to Static-MedRF, DeadlineGuard reduces energy by 43.7 J per run (2.98%; 95% paired CI 41.4–46.0 J) and eliminates a 10.24-pp deadline-miss rate, at a 1.373-pp F1 cost. Unlike the learned selectors, it realizes load-conditioned switching under the prospectively frozen factorial. This does not make DeadlineGuard universally optimal: Static-LightLR is still lower-energy and has the same zero deadline misses, while DeadlineGuard trades +7.55 J for a small 0.056-pp F1 gain relative to LightLR. The value of the comparison is that the high-security static baseline becomes infeasible at the declared service deadline and a simple causal rule exposes that trade-off explicitly.

The confidence cascade answers a different question. It recovers MedRF-like F1 using only 1.34% row escalation, but

TABLE VI
OFFLINE VALIDATION-SELECTED THRESHOLD DIAGNOSTIC FOR EVERY FROZEN TON-IOT DETECTOR. THE PHYSICAL CAMPAIGNS REMAIN FIXED AT $\tau = 0.5$.

Model	$\tau = 0.5$ F1	$\tau = 0.5$ FPR (%)	τ^*	τ^* F1	τ^* FPR (%)	Δ F1 (pp)	Validation BA
TinyDT	0.4792	67.78	0.646	0.9698	1.82	+49.05	0.9828
LightLR	0.9620	1.35	0.769	0.9605	1.22	-0.14	0.9899
MedRF	0.9746	1.52	0.572	0.9775	1.31	+0.29	0.9905
HeavyMLP	0.9736	1.38	0.681	0.9732	1.26	-0.04	0.9867

those rows occur in 66.76% of 100-row windows. The executed implementation performs one MedRF call per escalation-active window on a tiny deferred sub-batch, and that call has near-MedRF latency. Its observed whole-device cost is therefore specific to this call granularity and should not be read as a general limit on optimized batched cascades.

B. Reward Specification as a Pre-deployment Check

The policy collapse is not only an empirical observation. Equation (7) exposes whether the frozen scalar utility even permits a pair of actions to exchange rank on the declared support. For MedRF versus LightLR, candidate-range normalization implies a 0.7967 balanced-accuracy threshold—an almost 80-percentage-point exchange rate determined before held-out evaluation. A controller can therefore appear to “learn” a stable choice even when the declared objective makes reversal implausibly demanding on its calibration/validation support. The broader design lesson is to normalize resource terms against deployment contracts—idle-relative power, latency SLOs, energy budgets, or thermal headroom—rather than the arbitrary extrema of the current candidate pool.

C. Sparse Support and Off-support Safety

Tabular-Q’s full greedy map assigns 314/324 states to TinyDT and only ten to LightLR, while formal primary execution reaches four LightLR states. Frozen source confirms that this is not a separate runtime-exception fallback: unseen zero-initialized Q rows are tied, `np.argmax` returns the first maximum, and TinyDT is action index zero. variable-load campaign makes the consequence concrete: new load-related states drive the frozen table to TinyDT, increasing FPR by roughly 66 percentage points versus LightLR. The transferable risk is therefore the interaction of sparse support, unsafe default action ordering, and deterministic tie breaking. DQN maps the primary 324-state grid to LightLR, but that smoother off-support behavior is not evidence of validated generalization. A deployable discrete controller should report state coverage, the full action map, and the safety quality of unvisited/default actions—not only occupancy on visited states.

D. Measurement and Evaluation Implications

The variable-load campaign also illustrates why raw means alone are insufficient at small whole-device effects. DQN executes LightLR in every window, yet its raw energy difference from Static-LightLR is positive in 10/10 blocks

while the Student- t interval crosses zero because one thermal/power excursion dominates the variance. Reporting the block points, SD, median, exact sign-flip sensitivity, and preceding-idle adjustment exposes this instability; none licenses a causal controller-power explanation because the controller-only power study stopped before measurement.

A learned orchestrator should therefore be compared with the static action it realizes and with simple conditional controls when workload feasibility changes. Action occupancy, deadline violations, and confusion counts are indispensable: energy alone would favor the failed TinyDT collapse, while F1 alone would hide false alarms and service infeasibility. Raw and idle-adjusted physical endpoints answer different questions and neither is selected post hoc because it looks favorable.

The TinyDT diagnostic shows that its threshold-0.5 failure is substantially calibration-sensitive. That does not legitimize replacing the primary threshold after test inspection or treating TinyDT as safe in the already frozen physical campaigns. It instead motivates validation-only calibration governance before deployment. Generalization beyond this operating envelope still requires untouched traffic, additional boards, and independent metrology.

IX. LIMITATIONS AND REPRODUCIBILITY BOUNDARY

Classifier results use one training seed, dataset-native fixed splits, and threshold 0.5. Replay reuses held-out pools. Only TON-IoT receives dynamic physical testing; CICIoT2023 and N-BaIoT provide classification/replay evidence. Ten blocks on one Pi do not estimate board, meter, site, or fleet variability.

The KM003C is not laboratory calibrated; 1-Hz load-side USB measurements exclude AC conversion, network capture, and feature encoding. Models remain resident, temperatures stay low, and the factorial is scheduled feature replay rather than live packet arrival. Pairing reduces but cannot remove ambient drift, as the raw and preceding-idle-adjusted comparisons demonstrate. DQN raw energy in the variable-load campaign is highly skewed by one block with elevated preceding idle power and temperature; the direction is reported separately from the unstable magnitude. The fixed-rate cascade cost is specific to a 100-row windowing regime in which sparse deferred rows trigger a near-fixed-cost MedRF sub-batch call in 66.76% of windows.

The matched-control, overhead, and sensitivity analyses and the TinyDT threshold diagnostic are post-hoc to the primary campaign. The fixed-rate cascade and variable-load campaigns were prospectively frozen for their own execution. No equivalence margin was declared. The planned controller-only power study supplies no controller-power number because the frozen

four-state selection was ambiguous. The TinyDT threshold diagnostic assesses threshold sensitivity without replacing the fixed result. Invalid attempts and replacements remain archived; manifests, CRC, source hashes, and leakage validators establish package integrity, not external validity.

X. CONCLUSION

The experiments separate adaptive value from detector choice and controller implementation. Primary Tabular-Q appears cheaper than Static-MedRF only because it fixes LightLR; under expanded load, the same frozen table falls to a TinyDT default with 67.76% FPR, while DQN remains LightLR. In contrast, the simple causal DeadlineGuard switches between MedRF and LightLR, uses 43.7 J less than Static-MedRF (2.98%), and removes all 512 measured deadline misses at a 1.373-pp F1 cost. The pairwise non-degeneracy test explains how reward normalization can suppress action reversal before deployment, and the off-support analysis shows how zero-initialized tabular ties can expose an unsafe default action. Learned IDS orchestration should therefore be credited with adaptive value only after realized actions, relevant static and conditional controls, service feasibility, security loss, and whole-device cost are separated.

REFERENCES

- [1] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 2389–2401, 2024.
- [2] X. Zhang *et al.*, "Aoc-ids: Autonomous online framework with contrastive learning for intrusion detection," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024.
- [3] P. S. Bouzinis, P. Radoglou-Grammatikis, I. Makris, T. Lagkas, V. Argyriou, G. T. Papadopoulos, P. Sarigiannidis, and G. K. Karagiannidis, "Statavg: Mitigating data heterogeneity in federated learning for intrusion detection systems," *IEEE Transactions on Network and Service Management*, vol. 22, no. 4, pp. 2944–2955, 2025.
- [4] C. Zha *et al.*, "Dm-ids—a network intrusion detection method based on dual-modal fusion," *IEEE Transactions on Network and Service Management*, vol. 22, no. 4, pp. 3646–3661, 2025.
- [5] Y.-D. Lin, H.-X. Huang, D. Sudyana, and Y.-C. Lai, "Ai for ai-based intrusion detection as a service: Reinforcement learning to configure models, tasks, and capacities," *Journal of Network and Computer Applications*, vol. 229, p. 103936, 2024.
- [6] M. M. Hasan, R. Islam, Q. Mamun, M. Z. Islam, and J. Gao, "Ensemble-based adversarial defense with dynamic model selection for intrusion detection systems," *Internet of Things*, vol. 38, p. 101970, 2026.
- [7] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.
- [8] T. Bolukbasi, J. Wang, O. Dekel, and V. Saligrama, "Adaptive neural networks for efficient inference," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 527–536.
- [9] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *International Conference on Learning Representations*, 2018.
- [10] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 3301–3310.
- [11] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, 2017, pp. 613–627.
- [12] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Automated model-less inference serving," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 2021, pp. 397–411.
- [13] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like clockwork: Performance predictability from the bottom up," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 443–462.
- [14] F. W. Murti, S. Ali, G. Iosifidis, and M. Latva-aho, "Deep reinforcement learning for orchestrating cost-aware reconfigurations of vrans," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 200–216, 2024.
- [15] N. Wang *et al.*, "Intelligent driving task scheduling service in vehicle-edge collaborative networks based on deep reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4357–4368, 2024.
- [16] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu, "Dvfo: Learning-based dvfs for energy-efficient edge-cloud collaborative inference," *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9042–9059, 2024.
- [17] J. Lou, Z. Tang, W. Jia, W. Zhao, and J. Li, "Startup-aware dependent task scheduling with bandwidth constraints in edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 2, pp. 1586–1600, 2024.
- [18] M. Kim, J. Jang, Y. Choi, and H. J. Yang, "Distributed task offloading and resource allocation for latency minimization in mobile edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 15 149–15 166, 2024.
- [19] E. F. Maleki, W. Ma, L. Mashayekhy, and H. J. L. Roche, "Qos-aware content delivery in 5g-enabled edge computing: Learning-based approaches," *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9324–9336, 2024.
- [20] Z. Nan, Y. Han, S. Zhou, and Z. Niu, "Robust dnn partitioning and resource allocation under uncertain inference time," *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 3680–3696, 2026.
- [21] J. Wang, Z. Li, H. Liu, T. Qiu, and H. Luo, "A trust-based computation offloading framework in mobile cloud-edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 24, no. 6, pp. 5370–5385, 2025.
- [22] Z. Hu, J. Niu, T. Ren, X. Liu, and M. Guizani, "Sitoff: Enabling size-insensitive task offloading in d2d-assisted mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 24, no. 3, pp. 1567–1584, 2025.
- [23] Y. Zhang, W. Liang, Z. Xu, W. Xu, and M. Chen, "Aoi-aware inference services in edge computing via digital twin network slicing," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3154–3170, 2024.
- [24] Y. Luo, L. Pu, and C.-H. Liu, "Computing power and battery charging management for solar energy powered edge computing," *IEEE Transactions on Mobile Computing*, vol. 24, no. 3, pp. 1913–1927, 2025.
- [25] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [26] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [27] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>
- [28] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," arXiv:2108.13264, 2021. [Online]. Available: <https://arxiv.org/abs/2108.13264>
- [29] Canadian Institute for Cybersecurity, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," University of New Brunswick, accessed: Sep. 7, 2026. [Online]. Available: <https://www.unb.ca/cic/datasets/iiotdataset-2023.html>
- [30] NIST/SEMATECH, "Confidence limits for the mean," e-Handbook of Statistical Methods, Sec. 1.3.5.2, accessed: Sep. 7, 2026. [Online]. Available: <https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm>